

SINGLE RESPONSIBILITY PRINCIPLE

OPEN/CLOSED PRINCIPLE

ISKOV SUBSTITUTION PRINCIPLE

INTERFACE SEGREGATION PRINCIPLE

DEPENDENCY INVERSION PRINCIPLE

DIFFERENCE BETWEEN SOLID PRINCIPLES AND DESIGN PATTERNS

SOLID PRINCIPLES ARE THE RULES OR GUIDELINES TO MAKE YOUR CODE CLEANER AND MORE MAINTAINABLE, WHILE DESIGN PATTERNS ARE REUSABLE SOLUTIONS TO COMMON PROBLEMS IN SOFTWARE DESIGN.

THINK OF SOLID AS THE FOUNDATION THAT MAKES IT EASIER TO USE DESIGN PATTERNS.

SINGLE RESPONSIBILITY PRINCIPLE

WHAT IT IS: A CLASS SHOULD HAVE ONLY ONE REASON TO CHANGE, MEANING IT SHOULD ONLY DO ONE THING.

HOW TO USE: ENSURE EACH CLASS HANDLES ONLY ONE PART OF YOUR APPLICATION'S FUNCTIONALITY.

WHY IT'S NEEDED: REDUCES COMPLEXITY AND IMPROVES MAINTAINABILITY BY MINIMIZING THE IMPACT OF CHANGES.

OPEN/CLOSED PRINCIPLE

WHAT IT IS: SOFTWARE ENTITIES SHOULD BE OPEN FOR EXTENSION BUT CLOSED FOR MODIFICATION.

HOW TO USE: ADD NEW FUNCTIONALITY BY EXTENDING EXISTING CODE, NOT BY MODIFYING IT.

WHY IT'S NEEDED: HELPS PREVENT BUGS AND REGRESSION BY KEEPING TESTED CODE UNCHANGED.

ISKOV SUBSTITUTION PRINCIPLE

WHAT IT IS: IF THE PARENT CLASS PROMISES SOMETHING (LIKE FLYING), EVERY CHILD MUST BE ABLE TO DELIVER IT. IF THEY CAN'T, DON'T PUT IT IN THE PARENT - USE INTERFACES INSTEAD!

HOW TO USE: ENSURE THAT SUBCLASSES EXTEND THE BEHAVIOR OF THE PARENT CLASS WITHOUT ALTERING THE EXPECTED FUNCTIONALITY.

WHY IT'S NEEDED: ENSURES THAT POLYMORPHISM WORKS CORRECTLY AND PREVENTS INCORRECT ASSUMPTIONS WHEN SUBCLASSING.

THE RULE: ANY CHILD CLASS SHOULD WORK PERFECTLY WHEN USED IN PLACE OF ITS PARENT CLASS - NO EXCEPTIONS, NO SURPRISES, NO BREAKING CODE.

THE GOAL: WHEN YOU CREATE A SUBCLASS, EXTEND THE PARENT'S BEHAVIOR (MAKE IT BETTER), DON'T REPLACE IT WITH SOMETHING COMPLETELY DIFFERENT THAT BREAKS EXPECTATIONS.

REAL IMPACT: IF YOUR CODE WORKS WITH A "BIRD" CLASS, IT SHOULD WORK JUST AS WELL WITH ANY "SPARROW" OR "EAGLE" SUBCLASS - BUT NOT WITH A "PENGUIN" THAT THROWS ERRORS WHEN ASKED TO FLY.

BOTTOM LINE: DON'T PROMISE WHAT YOU CAN'T DELIVER - IF YOUR PARENT CLASS SAYS "ALL BIRDS CAN FLY," THEN EVERY BIRD SUBCLASS MUST ACTUALLY BE ABLE TO FLY, OR YOU'RE BREAKING LSP!

INTERFACE SEGREGATION PRINCIPLE

WHAT IT IS: CLIENTS SHOULD NOT BE FORCED TO IMPLEMENT INTERFACES THEY DON'T USE.

HOW TO USE: BREAK DOWN LARGE, GENERAL-PURPOSE INTERFACES INTO SMALLER, MORE SPECIFIC ONES

WHY IT'S NEEDED: PREVENTS CODE FROM BECOMING BLOATED AND REDUCES THE IMPLEMENTATION BURDEN FOR CLASSES.

DEPENDENCY INVERSION PRINCIPLE

WHAT IT IS: HIGH-LEVEL MODULES SHOULD NOT DEPEND ON LOW-LEVEL MODULES. BOTH SHOULD DEPEND ON ABSTRACTIONS

HOW TO USE: RELY ON INTERFACES OR ABSTRACT CLASSES RATHER THAN CONCRETE IMPLEMENTATIONS

WHY IT'S NEEDED: DECOUPLES THE HIGH-LEVEL AND LOW-LEVEL MODULES, MAKING YOUR SYSTEM MORE FLEXIBLE AND EASIER TO MAINTAIN.

- LOW-LEVEL CLASSES: HANDLE BASIC OPERATIONS SUCH AS FILE HANDLING, DATABASE CONNECTIONS, AND NETWORK COMMUNICATION.
- HIGH-LEVEL CLASSES: MANAGE COMPLEX BUSINESS LOGIC, COORDINATING AND DIRECTING LOW-LEVEL CLASSES TO PERFORM SPECIFIC TASKS.